Plan realizacji: desktopowy planer dnia studenta z asystentem AI

Czas realizacji: 5 tygodni

1. Cel projektu

Celem projektu jest stworzenie desktopowej aplikacji wspomagającej studenta w planowaniu dnia. Aplikacja łączy klasyczny planer (zadania, wydarzenia, terminy) z asystentem AI w formie chatbota, który potrafi nie tylko rozmawiać, ale też bezpośrednio modyfikować plan użytkownika oraz proponować harmonogram dnia.

2. Zakres funkcjonalny

Zakres podstawowy (MVP):

- zarządzanie zadaniami i wydarzeniami (dodawanie, edycja, usuwanie),
- widok dnia i tygodnia,
- czat z asystentem AI,
- wykonywanie akcji na planie przez asystenta (dodanie, przesunięcie, usunięcie zadania),
- generowanie planu dnia na podstawie priorytetów, terminów i dostępnego czasu.

Poza zakresem podstawowym (jeśli starczy czasu):

- preferencje użytkownika (godziny nauki, długość przerw),
- tryb ciemny,
- synchronizacja z zewnętrznym kalendarzem, powiadomienia, statystyki.

3. Technologie

Warstwa	Technologia
Interfejs użytkownika	React
Aplikacja desktopowa	Electron (opakowanie interfejsu, uruchamianie backendu)
Backend / API	Python, FastAPI
Baza danych	SQLite (lokalnie)

Warstwa	Technologia
Asystent AI	zewnętrzne API modelu językowego, wywoływanie funkcji (function calling)
Narzędzia	Git / GitHub, pytest, electron-builder, PyInstaller

4. Architektura

Aplikacja działa lokalnie na komputerze użytkownika. Proces główny Electrona uruchamia backend FastAPI w tle oraz okno z interfejsem React. Interfejs komunikuje się z backendem przez lokalne REST API. Backend zapisuje dane w SQLite i obsługuje moduł AI.

Moduł AI otrzymuje kontekst (zadania, terminy, preferencje) i zwraca odpowiedź tekstową lub wywołanie funkcji (np. dodanie zadania). Każde wywołanie jest walidowane po stronie backendu przed zapisem do bazy. Klucz API jest przechowywany wyłącznie po stronie backendu.

Struktura repozytorium (monorepo): `/frontend`, `/backend`, `/electron`, `/docs`.

5. Harmonogram

Tydzień 1: fundamenty

- utworzenie repozytorium, struktury katalogów, README,
- szkielet Electron + React + FastAPI, komunikacja przez całą ścieżkę ("hello world"),
- uruchamianie backendu z poziomu Electrona,
- model danych w SQLite: zadania, wydarzenia.

Rezultat: aplikacja desktopowa uruchamia się, interfejs komunikuje się z backendem.

Tydzień 2: rdzeń planera

- operacje CRUD na zadaniach i wydarzeniach (API i interfejs),
- widok dnia i tygodnia,
- pola: termin, priorytet, szacowany czas, status.

Rezultat: plan można w pełni prowadzić ręcznie, bez AI.

Tydzień 3: asystent AI

- endpoint czatu, integracja z API modelu językowego,
- interfejs czatu (historia rozmowy, strumieniowanie odpowiedzi),

- prompt systemowy i przekazywanie kontekstu (zadania na dziś i tydzień),
- function calling: dodawanie, przesuwanie i usuwanie zadań,
- walidacja danych zwracanych przez model.

Rezultat: asystent realnie modyfikuje plan użytkownika.

Tydzień 4: inteligentne planowanie

- generowanie planu dnia (priorytety, terminy, wolne sloty),
- obsługa przeplanowania ("nie zdążyłem"),
- preferencje użytkownika.

Rezultat: działa główna funkcja aplikacji, czyli automatyczne planowanie dnia.

Tydzień 5: stabilizacja i oddanie

- testy (jednostkowe backendu, scenariusze end-to-end),
- obsługa błędów API, limity zapytań, cache,
- ustawienia, tryb ciemny,
- spakowanie aplikacji do instalatora (PyInstaller + electron-builder),
- dokumentacja i przygotowanie demonstracji, bufor na poprawki.

Rezultat: gotowa, zainstalowana aplikacja i dokumentacja.

6. Ryzyka i sposoby ograniczenia

Ryzyko	Sposób ograniczenia
Koszt i limity API modelu	krótki kontekst, cache, ograniczenie liczby zapytań
Błędy modelu przy datach i zadaniach	walidacja po stronie backendu, potwierdzanie zmian przez użytkownika
Prywatność danych	dane przechowywane lokalnie, do API trafia tylko niezbędny kontekst
Opóźnienia harmonogramu	priorytetyzacja: w razie potrzeby rezygnacja z trybu ciemnego, preferencji i przeplanowania
Trudności z pakowaniem backendu	wczesny test pakowania w tygodniu 4

7. Kryteria ukończenia

- aplikacja uruchamia się jako program desktopowy bez ręcznego startu backendu,
- użytkownik może zarządzać zadaniami i wydarzeniami w widoku dnia i tygodnia,
- asystent AI potrafi dodać, przesunąć i usunąć zadanie na podstawie polecenia w czacie,
- funkcja "zaplanuj mi dzień" zwraca poprawny, zwalidowany harmonogram,
- projekt posiada dokumentację i podstawowe testy.